A Control Matrix for Scientific Variable-Role Probing

Anonymous Author(s)
Affiliation
Address
`email`

Abstract

Interpretability requires experiments that distinguish a model mechanism from properties of the measurement. We examine this problem in linear probes for five variable roles in code models. The study includes three models and collectively covers all seven programming languages in XLCoST, but not every model-language combination. High probe accuracy supports different conclusions under matched controls. Misleading names reduce index probe F1 by 0.272 yet increase accumulator F1 by 0.079. Iterator probes can be refit after the same intervention in two models. On Python, boolean occurrence type is decodable at 0.981–0.988 macro-F1, but a masked source-line classifier reaches 0.983 without a language model. Rerunning both predictors with aligned predictions gives paired problem-clustered deltas of -0.003 to $+0.002$ across the three models, with intervals excluding any probe advantage above 0.014–0.017. This provides a bounded negative result. Class or structure labels remain decodable after perturbation-specific refitting and pass a within-program label-permutation control in three models, yet a gate-specified activation patch at the query-name site fails in Qwen2.5-1.5B. These outcomes motivate a five-part control matrix. Decodability, probe-capacity control, cue sensitivity, surface sufficiency, and site-specific causal relevance are distinct claims, and clearing one does not imply clearing another. Reporting which comparisons were run, what each found, and which remain untested makes a probing claim falsifiable.

1 Introduction

Linear probes are often used to ask what information a representation contains. A successful probe establishes decodability under a particular readout. It does not by itself establish that the model computed an abstract feature, that the feature is unavailable in surface form, or that the model uses it behaviorally [Belinkov, 2022, Voita and Titov, 2020]. Control tasks constrain probe capacity [Hewitt and Liang, 2019], and targeted studies show that probe accuracy need not entail task relevance [Ravichander et al., 2021, Elazar et al., 2021]. Capacity is only one source of ambiguity. Input leakage, label construction, sample resolution, and an unmatched intervention can each preserve high accuracy while changing the scientific claim.

This distinction is central to interpretability as a science. A measurement is useful when its target is explicit, its alternative explanations are tested, and contrary outcomes can change the conclusion. We use variable-role probing as a comparative case study. Variable roles describe how an identifier functions in a program, such as accumulating values or indexing a container [Sajaniemi, 2002]. They are attractive probe targets because the same nominal role can be expressed by names, syntax, and distributed context. That multiplicity also makes probe accuracy underidentified. Prior code-probing studies, including the 15-task INSPECT framework, map decodable syntactic and semantic information across layers and models [Karmakar and Robbes, 2021, Troshin and Chirkova, 2022, Karmakar and Robbes, 2024]. We do not propose another taxonomy. We contribute a role-by-control

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Table 1: Model and language coverage. No row should be read as a complete three-model by seven-language experiment. Table 3 gives the comparison coverage for each role.

Role	Models	Language coverage
Accumulator	Qwen2.5-1.5B	Python plus Java, C++, C, C#
Index or key	Qwen2.5-1.5B	Python, Java, JavaScript, C++, C, C#
Iterator	Qwen2.5-Coder-1.5B, StarCoder2-7B	Python, Java, JavaScript, PHP, C++, C, C#
Boolean	All three	Python, Java, JavaScript, PHP
Class or structure	All three	Python, C++, JavaScript, C

evidence map that records which interpretation survives, which comparison is incommensurate, and what outcome would falsify each surviving claim.

Our evidence covers five roles, three models, and the seven-language XLCoST corpus [Zhu et al., 2022]. The experiment is intentionally reported as an uneven evidence matrix rather than a full factorial study. Qwen2.5-1.5B, Qwen2.5-Coder-1.5B, and StarCoder2-7B all appear in the controlled boolean and class or structure experiments [Qwen et al., 2024, Hui et al., 2024, Lozhkov et al., 2024]. Earlier accumulator and index runs use one model and a weaker split protocol. Iterator runs cover Qwen2.5-Coder-1.5B and StarCoder2-7B across all seven languages. This is a methodological case study of evidential boundaries, not a validation of the matrix as a universal framework. Its heterogeneity separates replicated findings from leads that still require confirmation.

The central result is not that one representation type wins. The same headline observation, high linear-probe F1, produces different evidential outcomes. Index probes depend strongly on names. Accumulator and iterator probes can be refit after misleading renames. A model-free classifier matches boolean probe scores, and paired intervals exclude a probe advantage above 0.017. Class or structure probe performance remains stable under tested renames and passes a randomized-label control, but the gate-specified causal intervention fails at one site. A single accuracy number cannot identify among these explanations.

2 Study design and evidence matrix

Roles, models, and languages. We treat five roles as related case studies, not measurement-equivalent instances of one construct. Accumulators update within loops. Indices or keys appear as loop counters or subscripts. Iterators are loop-bound names and their uses. Boolean labels distinguish assignment, conditional, loop, and return uses. Class or structure labels mark declared type names and references. Binary token probes label all other tokens as negative. Heuristic AST and regular-expression extractors vary by language and lack a common multilingual gold set. Protocols also differ. Accumulator, index, and iterator use legacy token-stratified splits and test-maximized layers. Boolean mean-pools occurrences, uses problem-grouped splits, and predicts usage type. The class or structure pipeline uses token labels, problem groups, and five seeds. The modern pipelines select layers on validation data. XLCoST contains C, C++, C#, Java, JavaScript, PHP, and Python. Table 1 states actual coverage. Scores across rows do not estimate one common task.

Four control questions. Renaming asks whether lexical identity carries the prediction. We compare original names with numeric, single-character, collapsed, random-noun, and misleading names drawn from another role. Each condition refits its probe and selects its layer, so preserved F1 means that a similarly predictive readout can be recovered after renaming. It does not show that one fixed feature direction is invariant. A model-free character classifier with the target name masked asks whether local source text already determines the label. Randomized-label controls ask whether the probe family can fit a specified null task. The boolean null assigns a random label per variable name, whereas the class or structure null permutes labels within programs. Activation patching asks whether replacing a residual state at a gate-specified site moves a behavioral logit readout [Meng et al., 2022].

These controls are not interchangeable. Renaming can show name robustness while leaving syntax untouched. With refitting, it measures recoverability rather than invariance. A positive selectivity score can reject its specified null task while leaving a surface shortcut intact. A model-free com-

Table 2: Probe results after the strongest available control. Values are macro-F1. Deltas compare the misleading-name condition with original names.

Role	Baseline probe	Key control result	Strongest supported inference
Index	0.959	$\Delta = -0.272$	Refit performance depends on names in the legacy Qwen run
Accumulator	0.893	$\Delta = +0.079$	Refit performance survives that rename in the legacy run
Iterator	0.988, 0.990	$\Delta = +0.012, +0.009$	Refit performance survives misleading names in two models
Boolean	0.981–0.988	paired $\Delta \in [-0.003, +0.002]$	Probe advantage over masked local source excluded above 0.017
Class or structure	0.979–0.982	$\|\Delta\| \leq 0.004$	Refit robustness plus within-program permutation selectivity

parator can establish surface sufficiency but cannot show how the model uses the feature. A causal intervention can ground a use claim only at its specified layer, span, direction, and readout.

Evidence grades. Boolean and class or structure experiments use problem-grouped splits, validation-selected layers, five seeds, and paired or clustered uncertainty. Iterator artifacts use two models but retain the legacy token-level split and test-maximized layer, without matched surface or capacity controls. Accumulator and index likewise use token-level splits, test-maximized layers, and whole-word renaming. We retain their large, opposite effects as a falsifiable hypothesis, not as a confirmed estimate of a population difference. A DeepSeek index run is excluded because a tokenizer offset error invalidates its activation alignment.

3 The same probe score supports different claims

3.1 Lexical intervention separates roles

Perturbation-specific refitting produces opposite effects for index and accumulator probes. The Qwen2.5-1.5B index probe falls from 0.959 to 0.687, a change of -0.272. The accumulator probe rises from 0.893 to 0.972, a change of $+0.079$. The intervention therefore does more than lower overall data quality. It changes how recoverable the two labels are under the legacy readout. This is the study's clearest dissociation, but occurrence-level splitting, test-maximized layers, and refitting prevent a fixed-direction invariance interpretation. This effect-size estimate remains exploratory.

The iterator experiments provide separate evidence of refit robustness at near-ceiling baseline accuracy. Qwen2.5-Coder-1.5B and StarCoder2-7B reach baseline F1 of 0.988 and 0.990. Misleading iterator names change those scores by $+0.012$ and $+0.009$. Python-trained probes transfer across the other six XLCoST languages with F1 ranges of 0.936–0.983 and 0.906–0.964. These results support recoverability after the tested naming intervention. They do not establish feature invariance or abstraction because the probe and best layer are refit and the iterator runs lack a matched model-free surface comparator.

3.2 A model-free baseline bounds the boolean result

Boolean occurrence type illustrates why a high, replicated probe score can still provide limited scientific evidence. On a frozen Python sample, the three models reach 0.982, 0.981, and 0.988 macro-F1. Selectivity ranges from $+0.272$ to $+0.288$. These values reject the specific hypothesis that an equally expressive probe predicts labels randomly assigned per variable name as well as it predicts the real usage labels. Yet a character classifier on the enclosing source line, with the variable name masked, reaches 0.983. The probe-minus-surface differences are -0.001, -0.002, and $+0.005$.

The original exports retained no aligned predictions, so we reran both predictors on the frozen sample and retained aligned predictions. Before pairing, the reruns reproduced the committed scores. The paired probe-minus-surface estimates, followed by their problem-clustered 95% intervals, are -0.002 $[-0.017, 0.014]$, -0.003 $[-0.037, 0.014]$, and $+0.002$ $[-0.016, 0.017]$ over 915 problem clusters. Every interval covers zero, and each excludes a probe advantage larger than 0.014–0.017 macro-F1.

Five renaming conditions also change refit performance by at most 0.013. The supported claim is now sharper. Boolean occurrence type is decodable after the tested renames. The paired intervals exclude a probe advantage above 0.014–0.017 over masked local syntax on Python. The cross-language runs extend decodability to Java, JavaScript, and PHP. Because only Python has renaming runs, the cross-language results do not extend the refit-robustness claim beyond Python.

3.3 A causal null limits the class or structure claim

The class or structure experiments and the boolean experiments use different controls. Across all three models, baseline F1 is 0.979–0.982 and within-program permutation selectivity is $+0.503$–$+0.553$. Misleading class-style names change F1 by -0.004, -0.004, and $+0.004$. Python-trained probes also transfer to C++, JavaScript, and C at 0.885–0.980. This supports a permutation-selective signal whose predictive performance can be recovered after the tested renames. It does not establish a fixed rename-invariant direction.

We then test a stronger causal claim in Qwen2.5-1.5B. The experiment contains 288 matched class-versus-function prompts in 48 template clusters. Let $D = \ell(\texttt{True}) - \ell(\texttt{False})$. Denoising patches the class-source residual into a function prompt and defines $e_d = D_{\text{patched function}} - D_{\text{function}}$. Noising reverses the patch and defines $e_n = D_{\text{class}} - D_{\text{patched class}}$. Recovery is the ratio of the mean signed effect to the mean matched gap $D_{\text{class}} - D_{\text{function}}$. Percentile intervals resample template clusters. The gate additionally requires positive, control-separated effects, recovery of at least 0.05, and mean effects of at least 0.10 in both directions.

Both prompt classes prefer `True`, which limits the behavioral readout's effective decision range. Their mean logit differences are $+2.75$ for class prompts and $+1.78$ for function prompts, although the matched class-minus-function gap is $+0.97$ and has the expected sign for 98.6% of pairs. The gate therefore tests whether patching transfers this relative distinction, not whether it flips a well-calibrated binary decision.

Denoising produces a mean effect of 0.0087 with a 95% interval $[-0.0007, 0.0188]$ and recovery 0.0089. Noising produces 0.0199 $[0.0109, 0.0296]$ with recovery 0.0204. The intervention therefore fails the specified gate. The protocol and results entered the repository together, so we do not claim auditable preregistration. This is a site-specific causal null, not evidence that class information is absent or unused everywhere. More directly, the intervals exclude mean effects above 0.0188 and 0.0296 logit-difference units in the two tested directions. Strong decodability, positive selectivity, and renaming robustness therefore do not establish a substantive bidirectional contribution at this site and readout.

4 A claim–control matrix for scientific probing

The experiments distinguish logically separate claims rather than a cumulative ladder.

1. **Decodability.** A held-out probe exceeds a simple label baseline.
2. **Probe-capacity control.** The probe exceeds a matched randomized-label task whose construction is reported. This implies neither training dependence nor surface irreducibility.
3. **Cue sensitivity.** A fixed probe on a paired rename isolates one question, whereas perturbation-specific refitting measures recoverability after the cue changes.
4. **Surface sufficiency.** A model-free comparator on the same population tests whether surface information already predicts the label, and a representational advantage further requires uncertainty on the paired difference.
5. **Site-specific causal relevance.** A gate-specified intervention changes a behavioral readout in both directions and separates from placebo and random controls.

No claim entails another. Table 3 records which comparisons were run for each role. Only boolean has a matched surface comparator, which finds masked local syntax sufficient. Only class or structure has a causal gate, which the intervention fails at the selected site.

Falsifying outcomes. The framework is useful only if results can change the interpretation. A problem-grouped rerun could shrink or reverse the accumulator-index dissociation. An all-layer class or structure patch could reveal a causal site outside the probe-selected layer. A seven-language

Table 3: Comparison coverage by role. Check marks denote modern comparisons, circles denote legacy comparisons awaiting replication, and NR denotes comparisons not run. Outcomes appear in the text and Table 4.

Comparison	Accum.	Index	Iterator	Boolean	Class
Decodability	○	○	○	✓	✓
Randomized-label control	NR	NR	NR	✓	✓
Rename intervention	○	○	○	✓	✓
Surface comparator	NR	NR	NR	✓	NR
Causal gate	NR	NR	NR	NR	✓

Table 4: Scientific status of the paper's main interpretations. "Open" means that the required comparison has not been run under a matched protocol.

Interpretation	Evidence in this study	Status and decisive next test
Legacy roles may use different cues	Opposite refit deltas in incommensurate pipelines, plus iterator stability	Exploratory. Repeat with problem groups, scope-aware renaming, and intervals
Boolean exceeds Python local syntax	Paired reruns, three models, 915 problem clusters	Not supported. 95% intervals exclude advantages above 0.014–0.017
Class refit performance survives renaming	Three-model permutation selectivity and misleading-name deltas near zero	Supported as recoverability. Evaluate one fixed probe across paired renames
Query-name signal makes a substantive bidirectional causal contribution	Two-direction patching with placebo and random controls	Not supported at Qwen layer 18. Run an explicitly exploratory all-layer sweep
Iterator exceeds local surface form	High F1 in two models with refit robustness, no model-free comparator	Open. Run a matched masked-source comparator on that sample
Cross-language role abstraction	Strong transfer with uneven role and model coverage	Open. Apply one extractor and control battery to a complete language grid

rename pipeline could reveal language-specific cue dependence. Each outcome would revise a named claim.

Replication standard. We recommend publishing the evidence matrix with the metric table. Every cell should identify the prediction unit, label, model, corpus, split, layer selection, seeds, comparator, uncertainty estimand, and intervention gate. Negative results should state the smallest effect the design can exclude.

5 Identifiability and evaluation design

Matched contrasts determine what each control identifies. The paired boolean intervals bound probe-minus-surface differences. Refit renaming measures recoverability. The class or structure patch fails the specified bidirectional gate at one site. None supports a broader invariance or causal claim.

6 Limitations

The study collectively covers three models, five roles, and seven languages, but not a complete $3 \times 5 \times 7$ design, and Table 1 gives the per-role model and language coverage.

Legacy accumulator-index results predate problem-grouped splits and scope-aware renaming, and function overlap, textual replacement, layer selection, and refitting may inflate the effect. Iterator has no matched surface or capacity comparator, and its legacy artifacts record only aggregate scores. Boolean labels are nearly predictable from local form, with paired 95% intervals whose upper endpoints do not exceed 0.017. Class or structure patching covers one model, layer, site, and readout, and stopping after the Qwen null was not registered in advance. Other probes remain correlational, and cross-language transfer may reflect shared syntax, names, or extractor artifacts.

References

Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022. doi: 10.1162/coli_a_00422.

Yanai Elazar, Shauli Ravfogel, Alon Jacovi, and Yoav Goldberg. Amnesic probing: Behavioral explanation with amnesic counterfactuals. *Transactions of the Association for Computational Linguistics*, 9:160–175, 2021. doi: 10.1162/tacl_a_00359.

John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2733–2743, Hong Kong, China, 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1275.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. doi: 10.48550/arXiv.2409.12186.

Anjan Karmakar and Romain Robbes. What do pre-trained code models know about code? In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1332–1336, Melbourne, Australia, 2021. IEEE. doi: 10.1109/ASE51524.2021.9678927.

Anjan Karmakar and Romain Robbes. INSPECT: Intrinsic and systematic probing evaluation for code transformers. *IEEE Transactions on Software Engineering*, 50(2):220–238, 2024. doi: 10.1109/TSE.2023.3341624.

Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauss, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Han Hu, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Munoz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. StarCoder 2 and The Stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024. doi: 10.48550/arXiv.2402.19173.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. *Advances in Neural Information Processing Systems*, 35, 2022. doi: 10.52202/068431-1262.

Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Abhilasha Ravichander, Yonatan Belinkov, and Eduard Hovy. Probing the probing paradigm: Does probing accuracy entail task relevance? In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 3363–3377, Online, 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.eacl-main.295.

Jorma Sajaniemi. An empirical analysis of roles of variables in novice-level procedural programs. In *Proceedings IEEE 2002 Symposia on Human Centric Computing Languages and Environments*, pages 37–39, Arlington, VA, USA, 2002. IEEE Computer Society. doi: 10.1109/HCC.2002.1046340.

Sergey Troshin and Nadezhda Chirkova. Probing pretrained models of source codes. In *Proceedings of the Fifth BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP*, pages 371–383, Abu Dhabi, United Arab Emirates (Hybrid), December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.blackboxnlp-1.31. URL `https://aclanthology.org/2022.blackboxnlp-1.31/`.

Elena Voita and Ivan Titov. Information-theoretic probing with minimum description length. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. doi: 10.18653/v1/2020.emnlp-main.14.

Ming Zhu, Aneesh Jain, Karthik Suresh, Roshan Ravindran, Sindhu Tipirneni, and Chandan K. Reddy. XLCoST: A benchmark dataset for cross-lingual code intelligence. *arXiv preprint arXiv:2206.08474*, 2022. doi: 10.48550/arXiv.2206.08474.

A Per-role control results

Table 5: Complete boolean occurrence-type probe and model-free baseline results. The best baseline is selected from name-only, masked statement, masked line, masked window, covariate-only, and majority features. ρ is a descriptive one-instance score step, not an interval or equivalence test. Aligned probe/baseline predictions were not retained.

language	model	problems	probe F1	best baseline	difference	ρ	selectivity
Java	Qwen2.5-1.5B	1,427	0.982	0.980	+0.002	0.0120	+0.255
Java	Qwen2.5-Coder-1.5B	1,427	0.976	0.980	-0.004	0.0079	+0.271
Java	StarCoder2-7B	1,427	0.990	0.980	+0.010	0.0120	+0.243
JavaScript	Qwen2.5-1.5B	1,351	0.989	0.992	-0.003	0.0170	+0.132
JavaScript	Qwen2.5-Coder-1.5B	1,351	0.998	0.992	+0.006	0.0170	+0.150
JavaScript	StarCoder2-7B	1,351	0.988	0.992	-0.004	0.0170	+0.120
PHP	Qwen2.5-1.5B	1,305	0.983	0.969	+0.014	0.0080	+0.214
PHP	Qwen2.5-Coder-1.5B	1,305	0.975	0.969	+0.006	0.0080	+0.216
PHP	StarCoder2-7B	1,305	0.990	0.969	+0.021	0.0080	+0.240
Python	Qwen2.5-1.5B	1,301	0.982	0.983	-0.001	0.0068	+0.280
Python	Qwen2.5-Coder-1.5B	1,301	0.981	0.983	-0.002	0.0068	+0.272
Python	StarCoder2-7B	1,301	0.988	0.983	+0.005	0.0068	+0.288

Table 6: Boolean Python identifier-renaming audit. C1–C5 are the five committed renaming conditions. Each condition refits a probe and reselects its layer, so deltas measure recoverability rather than fixed-direction invariance. Intervals are problem-clustered. Empty renaming fields for other languages indicate that those runs were not committed, not a null result.

model	C1	C2	C3	C4	C5
Qwen2.5-1.5B	+0.003	-0.004	+0.001	-0.003	+0.001
Qwen2.5-Coder-1.5B	+0.001	-0.010	+0.006	-0.004	-0.006
StarCoder2-7B	-0.003	-0.013	-0.006	-0.001	-0.006

Table 7: Complete model-free boolean baselines by language. Baselines are shared across the three neural models because they use the same frozen examples and surface features.

language	majority	name	statement	line	window	covariates
Java	0.311	0.522	0.980	0.501	0.662	0.311
JavaScript	0.227	0.373	0.992	0.369	0.552	0.226
PHP	0.228	0.505	0.969	0.413	0.581	0.228
Python	0.217	0.422	0.970	0.983	0.605	0.216

Boolean probe figures. The baseline panels retain the individual surface comparators hidden by the best-baseline table. Renaming is available only for Python. The absence of corresponding figures in other languages is a coverage hole.

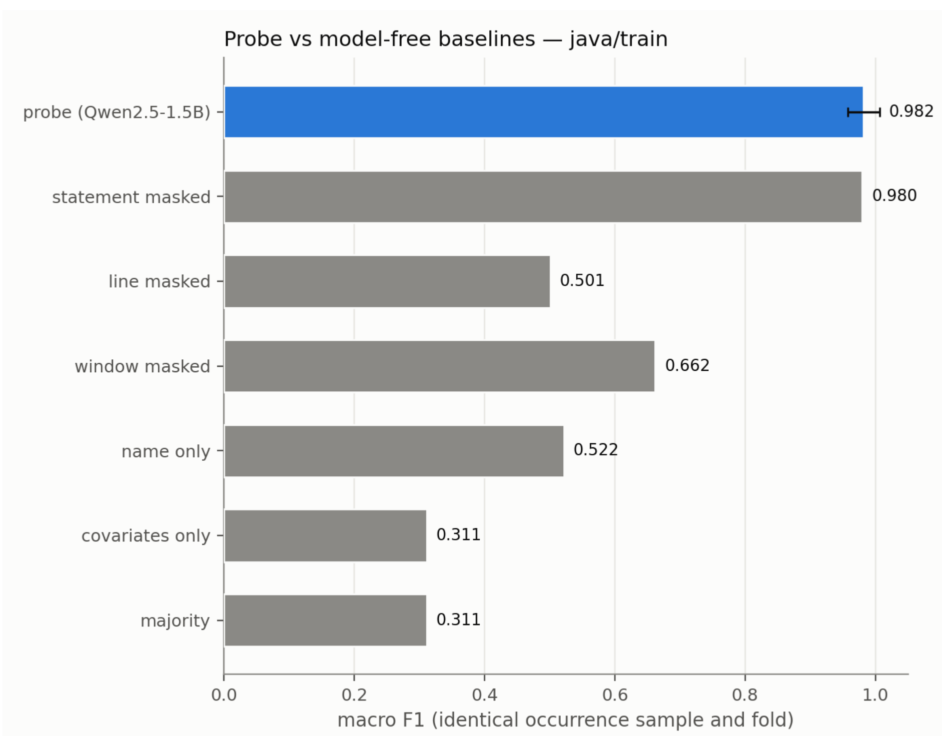

Figure 1: Boolean probe and model-free baselines for Java, Qwen2.5-1.5B.

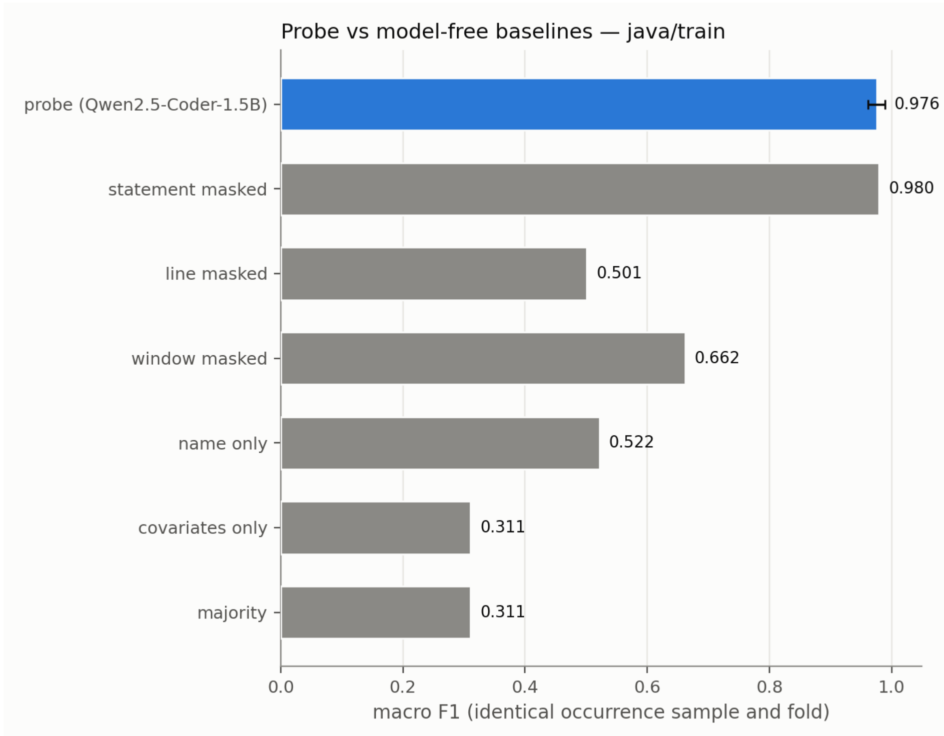

Figure 2: Boolean probe and model-free baselines for Java, Qwen2.5-Coder-1.5B.

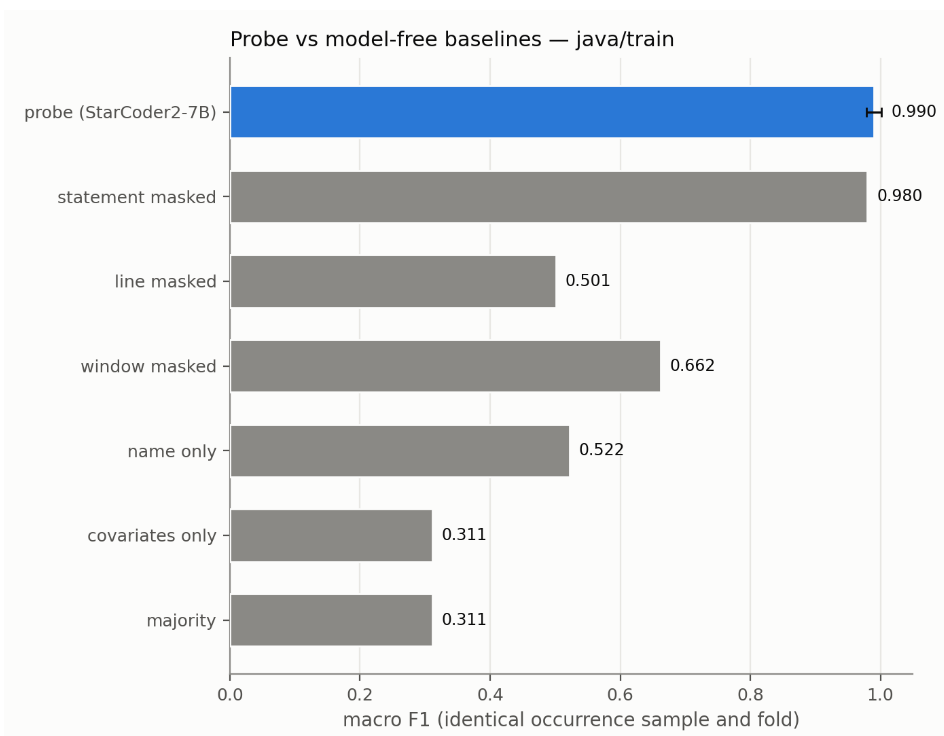

Figure 3: Boolean probe and model-free baselines for Java, StarCoder2-7B.

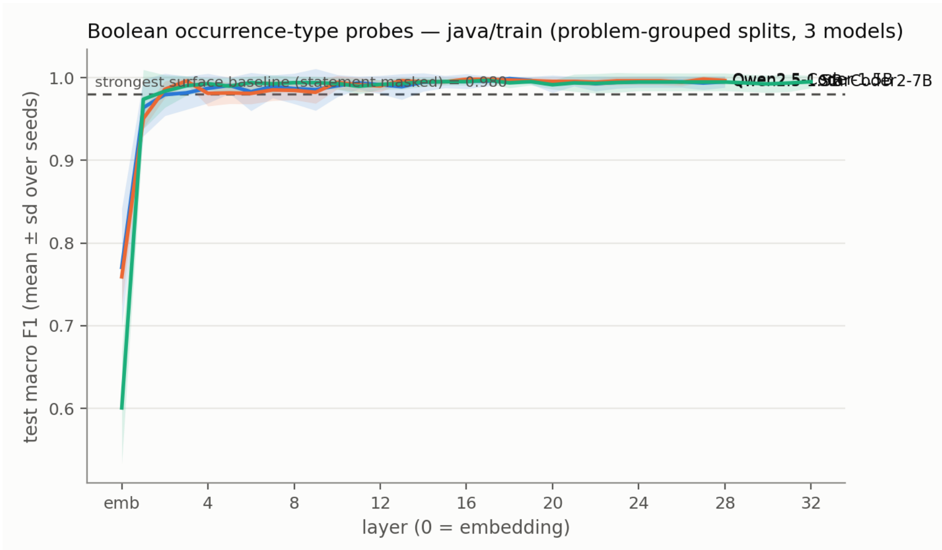

Figure 4: Boolean probe layer curves for Java across the three models.

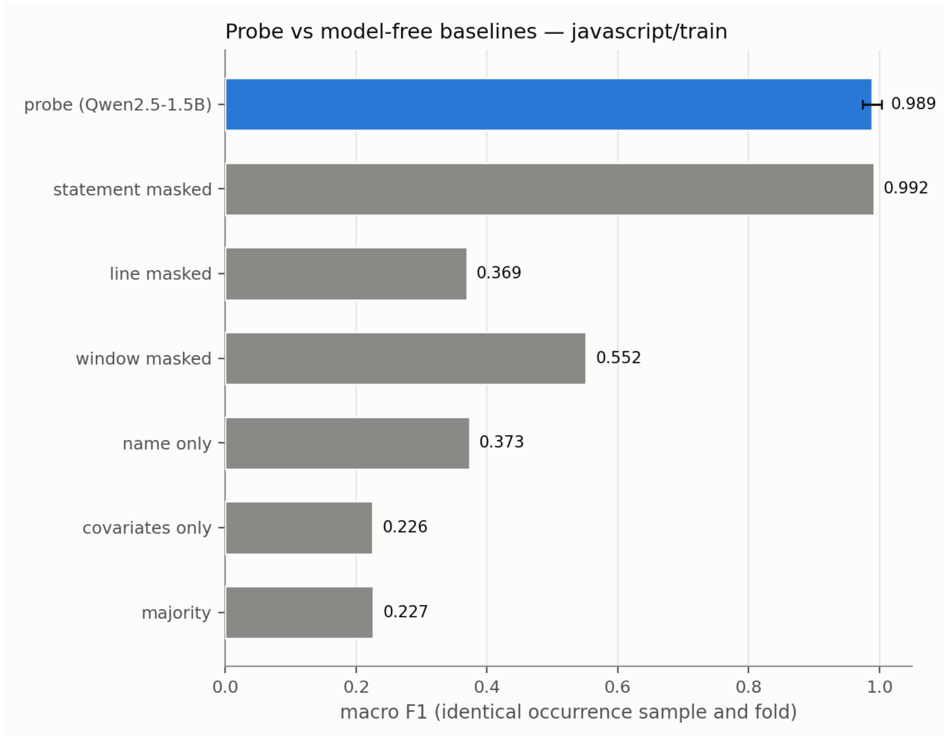

Figure 5: Boolean probe and model-free baselines for JavaScript, Qwen2.5-1.5B.

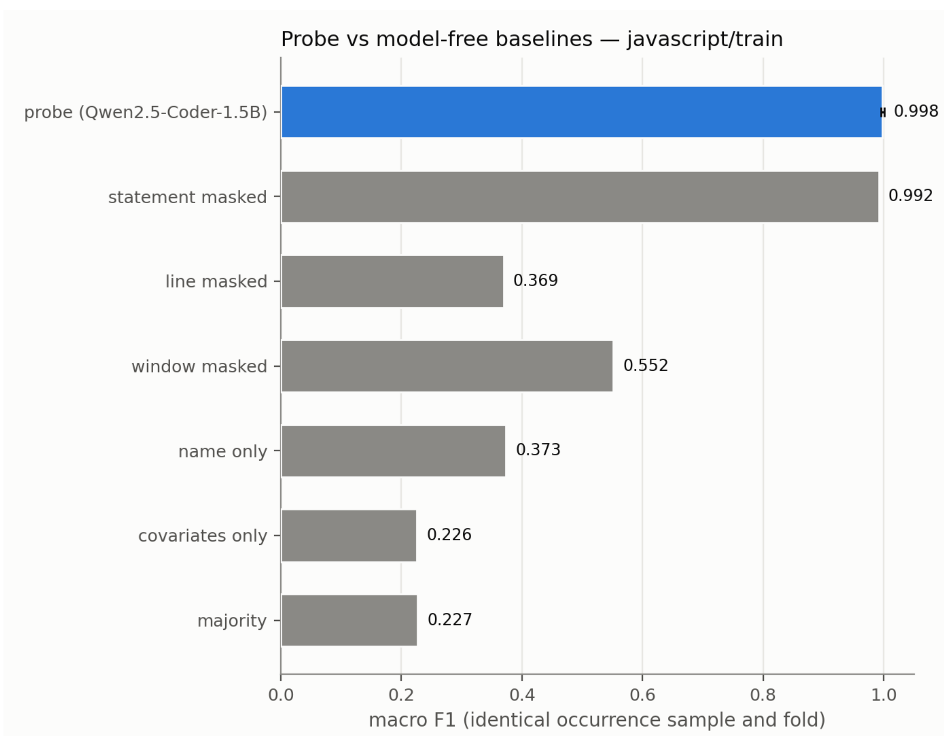

Figure 6: Boolean probe and model-free baselines for JavaScript, Qwen2.5-Coder-1.5B.

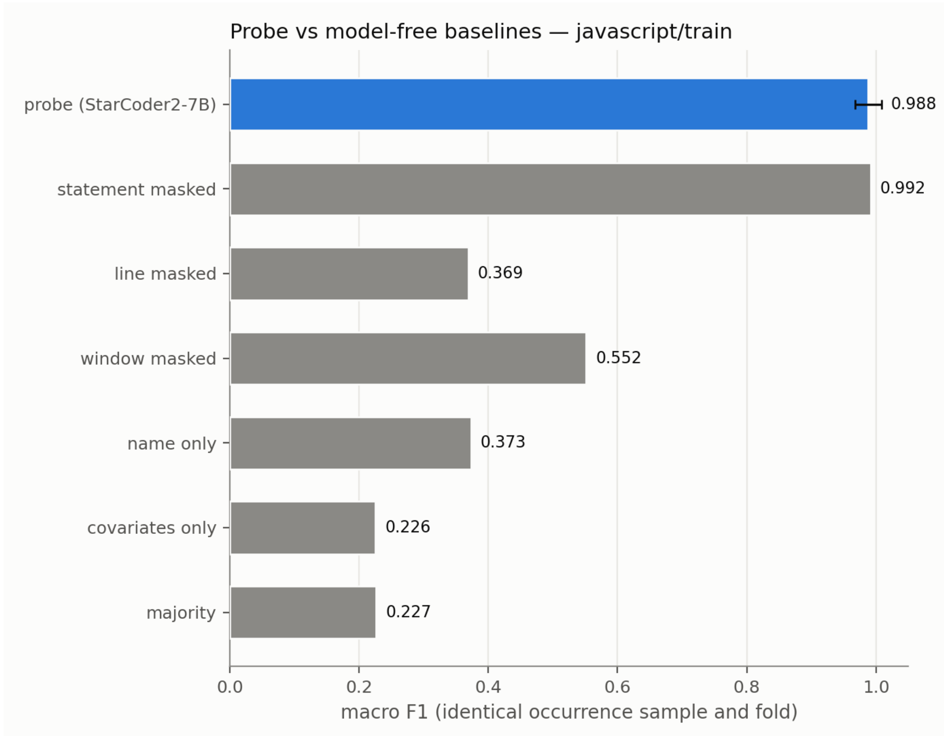

Figure 7: Boolean probe and model-free baselines for JavaScript, StarCoder2-7B.

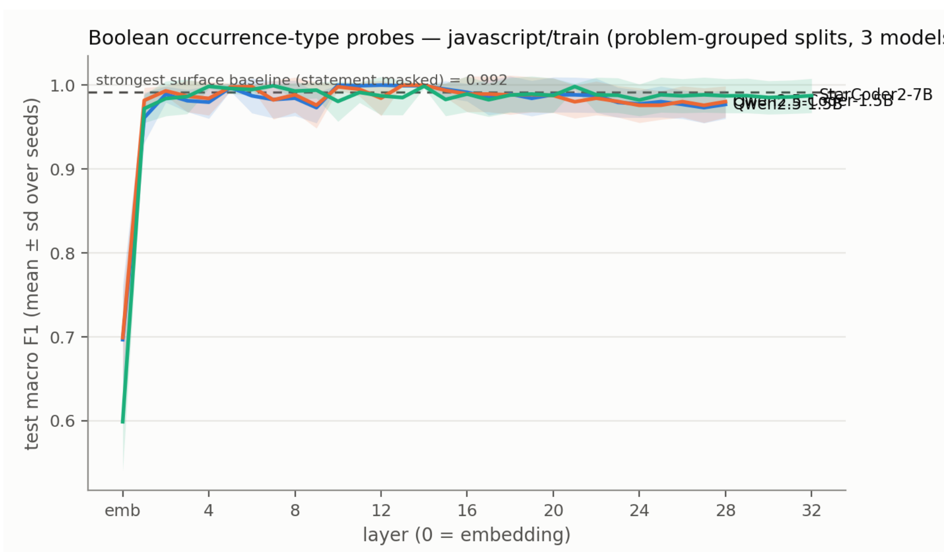

Figure 8: Boolean probe layer curves for JavaScript across the three models.

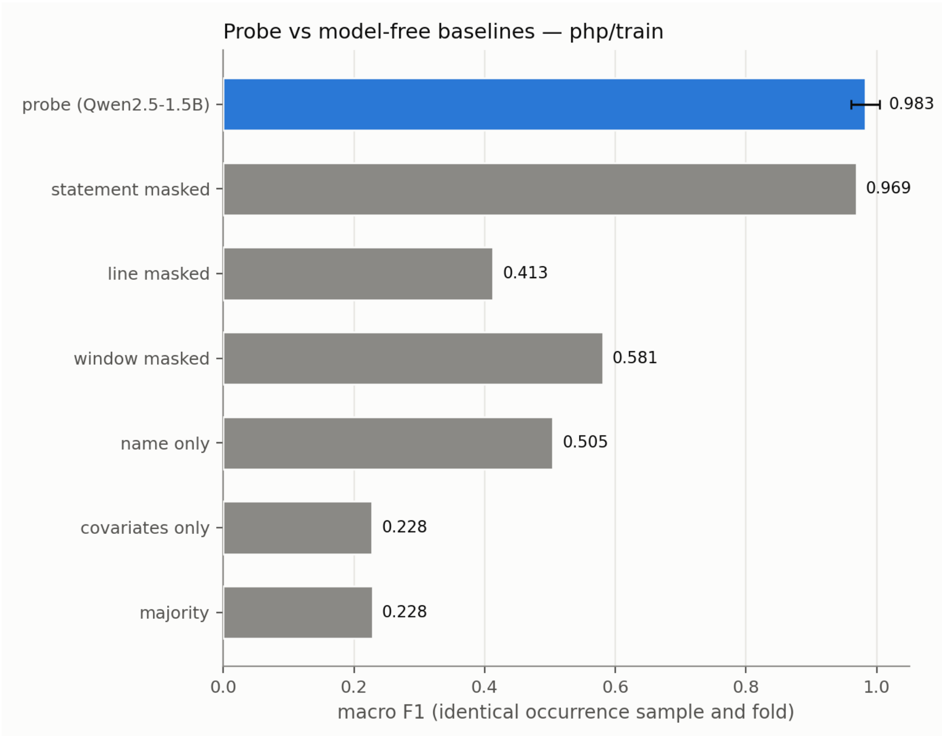

Figure 9: Boolean probe and model-free baselines for PHP, Qwen2.5-1.5B.

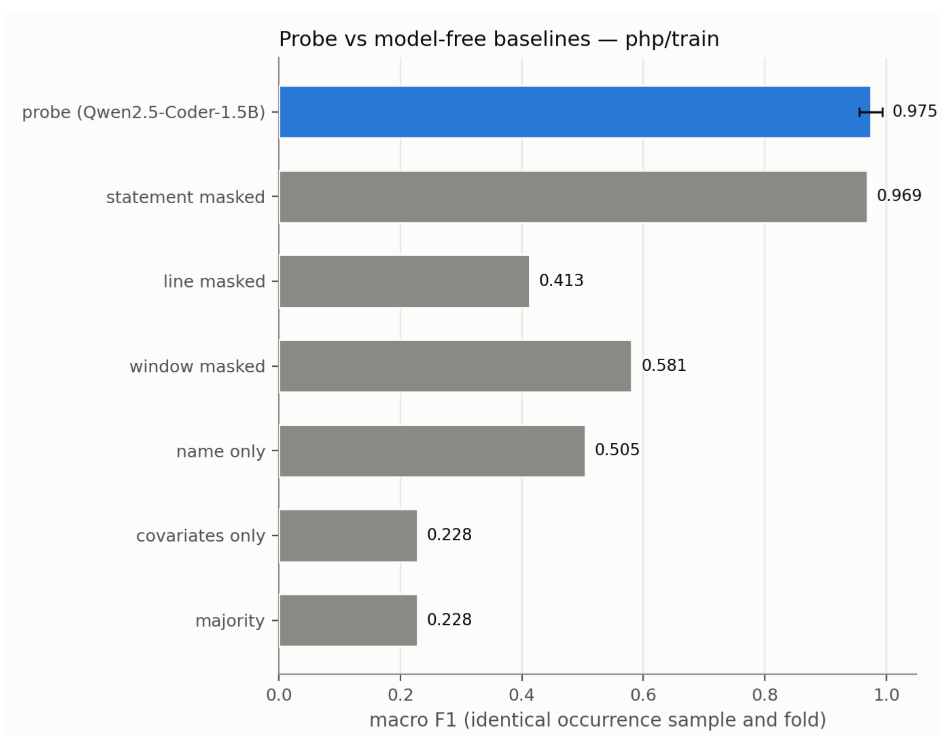

Figure 10: Boolean probe and model-free baselines for PHP, Qwen2.5-Coder-1.5B.

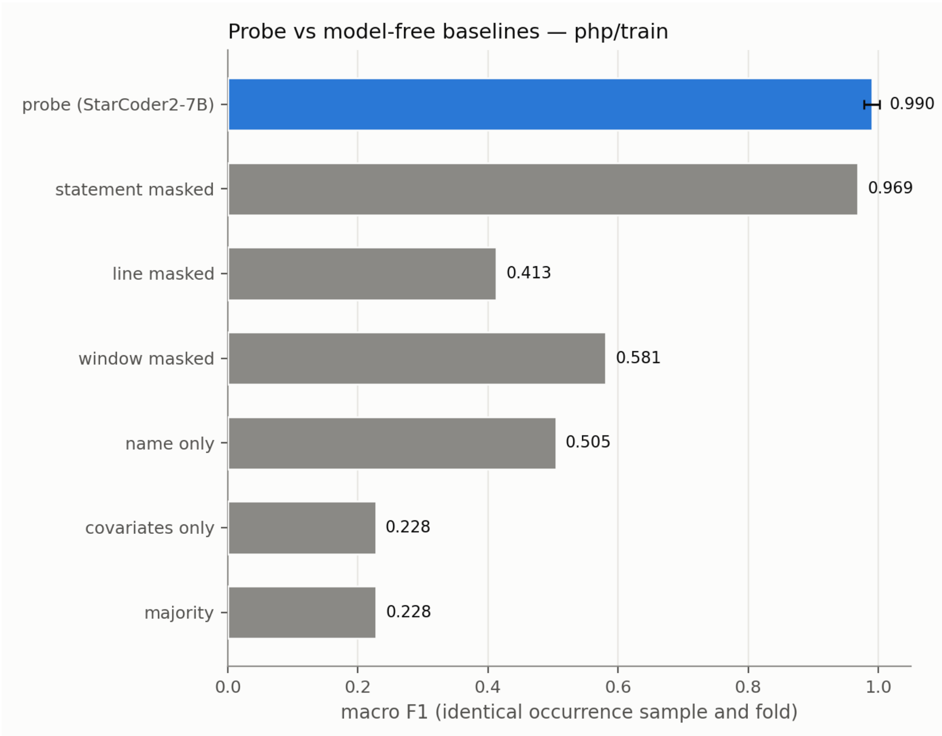

Figure 11: Boolean probe and model-free baselines for PHP, StarCoder2-7B.

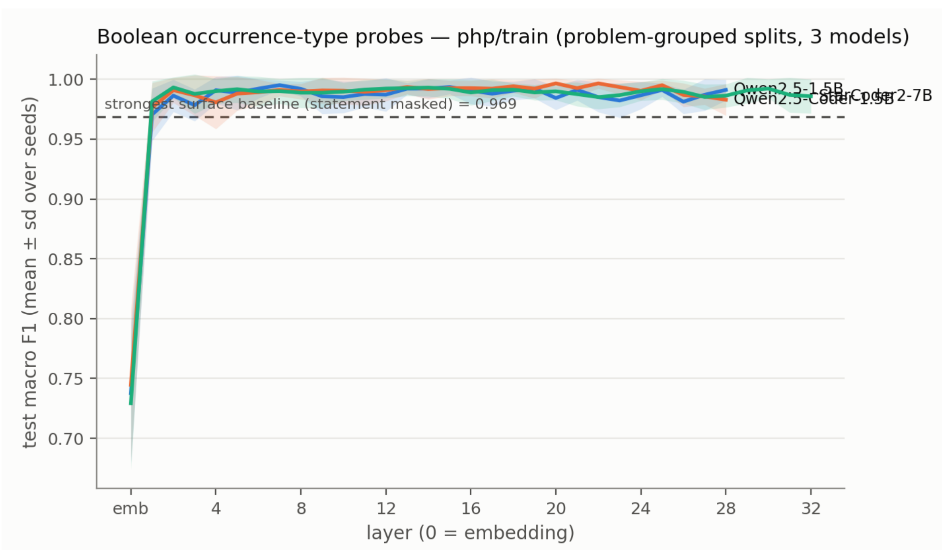

Figure 12: Boolean probe layer curves for PHP across the three models.

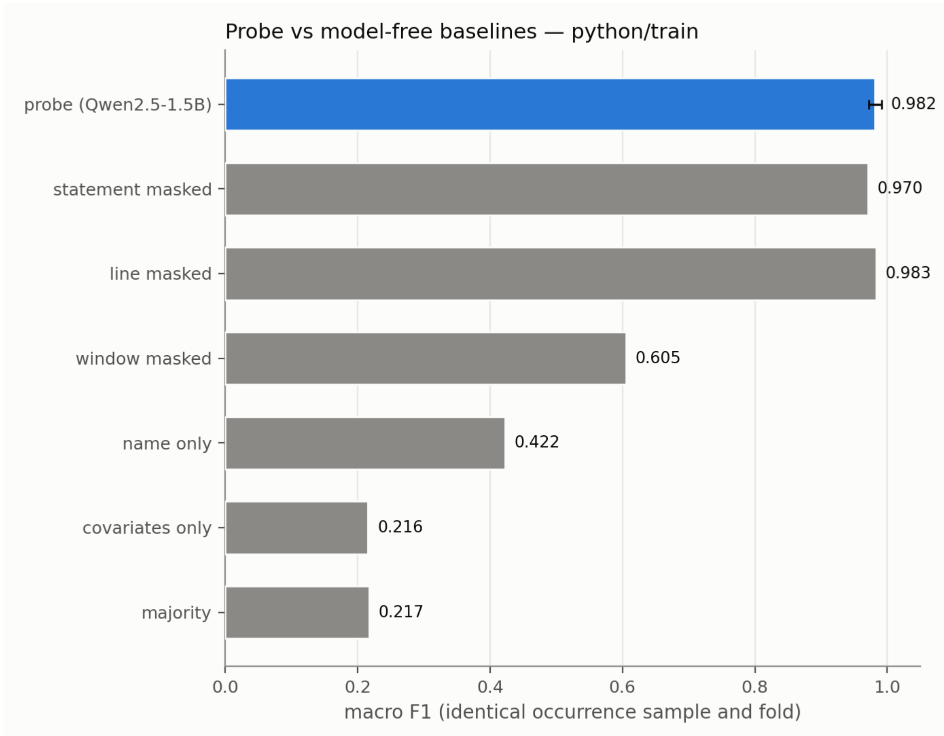

Figure 13: Boolean probe and model-free baselines for Python, Qwen2.5-1.5B.

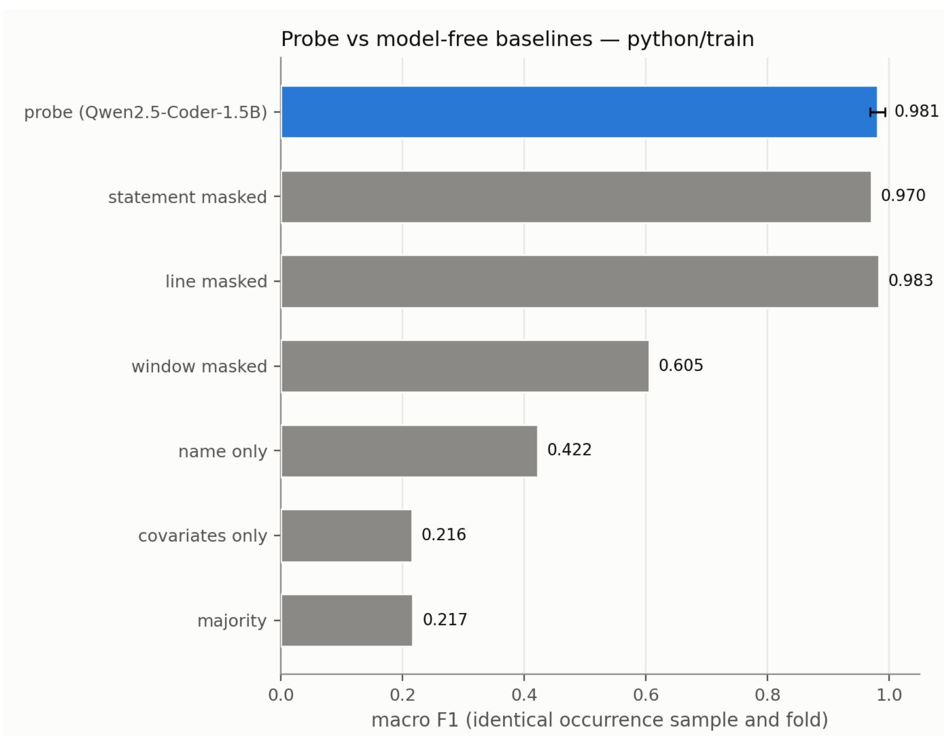

Figure 14: Boolean probe and model-free baselines for Python, Qwen2.5-Coder-1.5B.

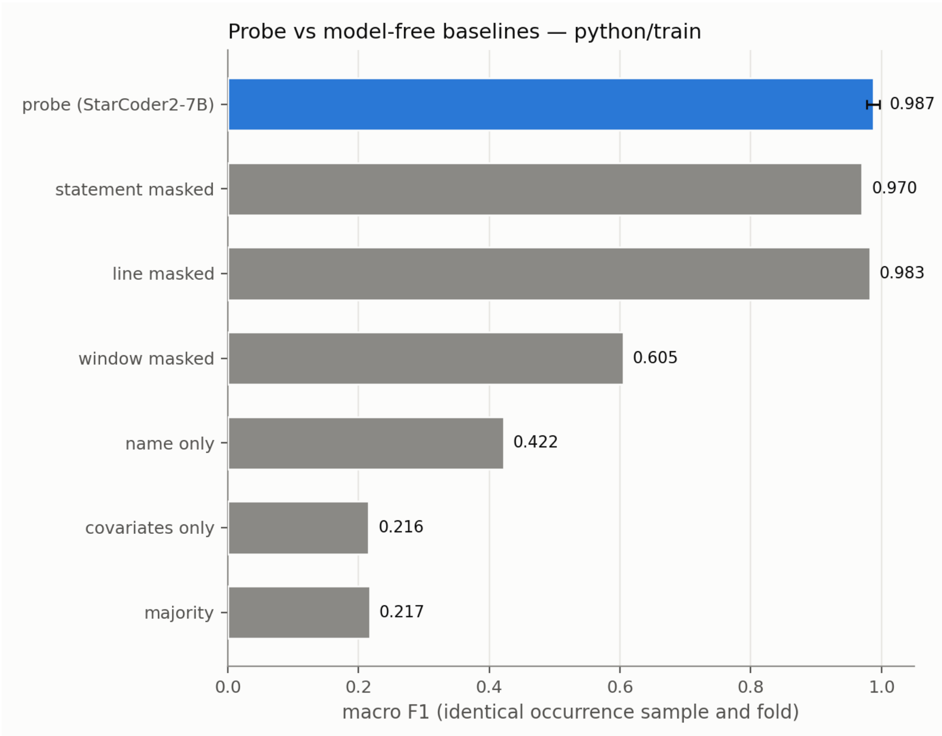

Figure 15: Boolean probe and model-free baselines for Python, StarCoder2-7B.

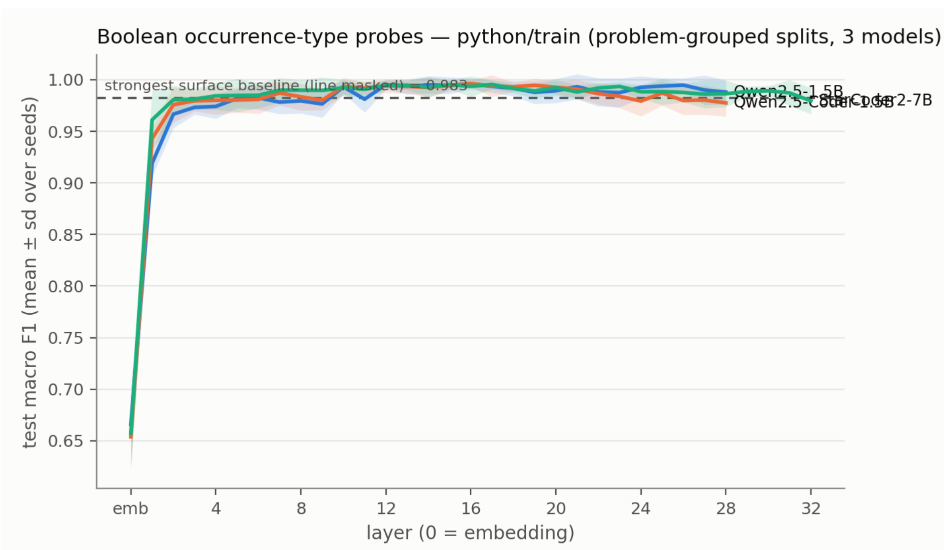

Figure 16: Boolean probe layer curves for Python across the three models.

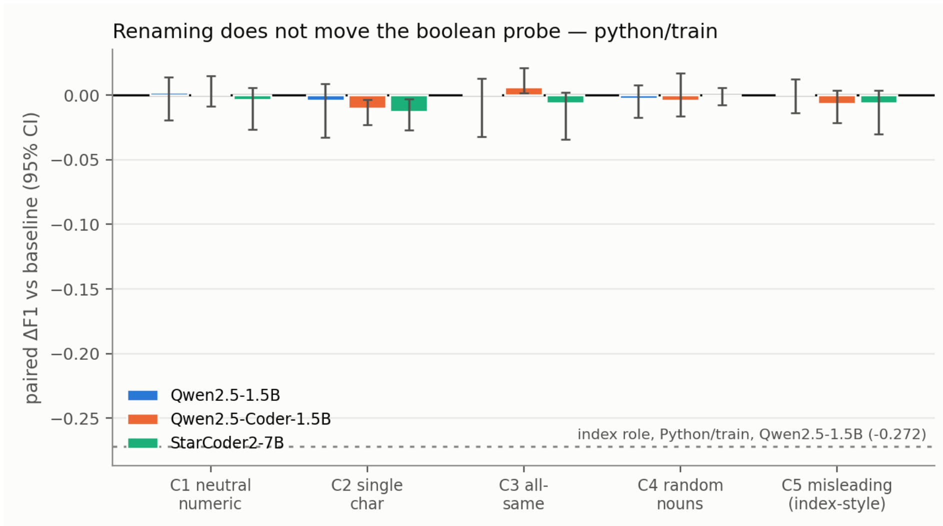

Figure 17: Problem-paired boolean refit-performance deltas on Python. No corresponding committed renaming run exists for other languages.

Table 8: Complete class/structure perturbation results under the shared five-seed protocol. Each condition refits a probe and selects its own layer on validation data, so cross-condition deltas do not test a fixed direction.

model	strategy	layer	F1	95% CI	control F1	selectivity	Δ
Qwen2.5-1.5B	baseline	L18	0.979	[0.967, 0.987]	0.427	+0.552	N/A
Qwen2.5-1.5B	random_nouns	L20	0.986	[0.976, 0.992]	0.426	+0.560	+0.007
Qwen2.5-1.5B	single_chars	L11	0.957	[0.936, 0.969]	0.423	+0.534	-0.022
Qwen2.5-1.5B	all_same	L0	1.000	[1.000, 1.000]	0.440	+0.560	+0.021
Qwen2.5-1.5B	numeric_vars	L21	0.965	[0.952, 0.972]	0.422	+0.543	-0.014
Qwen2.5-1.5B	misleading_class_struct	L7	0.976	[0.962, 0.982]	0.416	+0.559	-0.004
Qwen2.5-Coder-1.5B	baseline	L8	0.982	[0.969, 0.989]	0.429	+0.553	N/A
Qwen2.5-Coder-1.5B	random_nouns	L19	0.978	[0.965, 0.985]	0.427	+0.551	-0.004
Qwen2.5-Coder-1.5B	single_chars	L7	0.956	[0.936, 0.967]	0.423	+0.533	-0.026
Qwen2.5-Coder-1.5B	all_same	L0	1.000	[1.000, 1.000]	0.440	+0.560	+0.018
Qwen2.5-Coder-1.5B	numeric_vars	L21	0.968	[0.955, 0.975]	0.424	+0.544	-0.014
Qwen2.5-Coder-1.5B	misleading_class_struct	L7	0.978	[0.967, 0.984]	0.415	+0.562	-0.004
StarCoder2-7B	baseline	L5	0.981	[0.967, 0.990]	0.478	+0.503	N/A
StarCoder2-7B	random_nouns	L6	0.976	[0.962, 0.983]	0.466	+0.510	-0.006
StarCoder2-7B	single_chars	L4	0.957	[0.921, 0.971]	0.475	+0.482	-0.025
StarCoder2-7B	all_same	L0	1.000	[1.000, 1.000]	0.445	+0.555	+0.019
StarCoder2-7B	numeric_vars	L23	0.968	[0.956, 0.975]	0.451	+0.517	-0.013
StarCoder2-7B	misleading_class_struct	L5	0.985	[0.976, 0.990]	0.467	+0.518	+0.004

Table 9: Class/structure cross-language results. Transfer uses the Python-selected layer. In-domain layers are selected separately. Only languages present in the committed exports are shown.

model	language	programs	in-domain layer	in-domain F1	transfer accuracy	transfer F1
Qwen2.5-1.5B	Python	400	L18	0.979	N/A	N/A
Qwen2.5-1.5B	C++	606	L10	0.991	0.993	0.933
Qwen2.5-1.5B	JavaScript	285	L5	0.987	0.998	0.957
Qwen2.5-1.5B	C	97	L8	0.986	0.984	0.886
Qwen2.5-Coder-1.5B	Python	400	L8	0.982	N/A	N/A
Qwen2.5-Coder-1.5B	C++	606	L8	0.988	0.991	0.918
Qwen2.5-Coder-1.5B	JavaScript	285	L6	0.985	0.996	0.913
Qwen2.5-Coder-1.5B	C	97	L20	0.980	0.983	0.885
StarCoder2-7B	Python	400	L5	0.981	N/A	N/A
StarCoder2-7B	C++	606	L5	0.992	0.996	0.965
StarCoder2-7B	JavaScript	285	L3	0.989	0.999	0.980
StarCoder2-7B	C	97	L5	0.986	0.987	0.906

Table 10: Layerwise class/structure robustness summaries. The F1 range is taken over all committed layers. Maximum cosine similarity is measured against the baseline activation direction for each renamed condition.

model	strategy	minimum F1	maximum F1 (layer)	maximum cosine (layer)
Qwen2.5-1.5B	all_same	0.999	1.000 (L0)	0.105 (L11)
Qwen2.5-1.5B	baseline	0.806	0.983 (L18)	N/A
Qwen2.5-1.5B	misleading_class_struct	0.708	0.980 (L7)	0.311 (L15)
Qwen2.5-1.5B	numeric_vars	0.445	0.966 (L21)	0.329 (L22)
Qwen2.5-1.5B	random_nouns	0.529	0.989 (L15)	0.493 (L26)
Qwen2.5-1.5B	single_chars	0.530	0.965 (L7)	0.392 (L15)
Qwen2.5-Coder-1.5B	all_same	0.999	1.000 (L0)	0.126 (L28)
Qwen2.5-Coder-1.5B	baseline	0.807	0.985 (L7)	N/A
Qwen2.5-Coder-1.5B	misleading_class_struct	0.708	0.978 (L7)	0.324 (L7)
Qwen2.5-Coder-1.5B	numeric_vars	0.445	0.969 (L24)	0.352 (L22)
Qwen2.5-Coder-1.5B	random_nouns	0.530	0.983 (L23)	0.464 (L19)
Qwen2.5-Coder-1.5B	single_chars	0.530	0.959 (L7)	0.415 (L15)
StarCoder2-7B	all_same	0.999	1.000 (L0)	0.101 (L21)
StarCoder2-7B	baseline	0.763	0.984 (L6)	N/A
StarCoder2-7B	misleading_class_struct	0.717	0.985 (L3)	0.387 (L3)
StarCoder2-7B	numeric_vars	0.443	0.971 (L26)	0.369 (L22)
StarCoder2-7B	random_nouns	0.502	0.979 (L8)	0.511 (L3)
StarCoder2-7B	single_chars	0.529	0.962 (L3)	0.467 (L6)

Table 11: Complete iterator perturbation results. Each condition refits a probe and selects its own best layer. These single-run summaries do not carry the five-seed intervals used for the class/structure role.

model	strategy	best layer	accuracy	F1	Δ
Qwen2.5-Coder-1.5B	baseline	L6	0.998	0.988	N/A
Qwen2.5-Coder-1.5B	random_nouns	L9	0.996	0.974	-0.014
Qwen2.5-Coder-1.5B	single_chars	L9	0.985	0.923	-0.064
Qwen2.5-Coder-1.5B	all_same	L26	1.000	1.000	+0.012
Qwen2.5-Coder-1.5B	numeric_vars	L23	0.978	0.924	-0.064
Qwen2.5-Coder-1.5B	misleading_iterator	L3	1.000	1.000	+0.012
StarCoder2-7B	baseline	L5	0.999	0.990	N/A
StarCoder2-7B	random_nouns	L7	0.997	0.986	-0.005
StarCoder2-7B	single_chars	L5	0.993	0.961	-0.029
StarCoder2-7B	all_same	L4	1.000	1.000	+0.009
StarCoder2-7B	numeric_vars	L24	0.989	0.958	-0.032
StarCoder2-7B	misleading_iterator	L4	1.000	0.999	+0.009

Table 12: Complete iterator cross-language transfer results. Transfer columns use the Python-selected layer.

model	language	programs	in-domain layer	in-domain F1	transfer accuracy	transfer F1
Qwen2.5-Coder-1.5B	Python	256	L7	0.988	N/A	N/A
Qwen2.5-Coder-1.5B	Java	188	L12	0.996	0.991	0.950
Qwen2.5-Coder-1.5B	C++	191	L7	0.994	0.994	0.967
Qwen2.5-Coder-1.5B	C	191	L12	0.991	0.988	0.936
Qwen2.5-Coder-1.5B	C#	189	L13	0.998	0.992	0.957
Qwen2.5-Coder-1.5B	JavaScript	266	L12	0.992	0.995	0.978
Qwen2.5-Coder-1.5B	PHP	253	L8	0.995	0.997	0.983
StarCoder2-7B	Python	256	L10	0.991	N/A	N/A
StarCoder2-7B	Java	188	L3	0.997	0.989	0.929
StarCoder2-7B	C++	191	L5	0.997	0.991	0.941
StarCoder2-7B	C	191	L16	0.992	0.984	0.906
StarCoder2-7B	C#	189	L11	0.995	0.988	0.923
StarCoder2-7B	JavaScript	266	L16	0.998	0.986	0.931
StarCoder2-7B	PHP	253	L17	0.998	0.993	0.964

Table 13: Layerwise iterator robustness summaries. F1 ranges cover every committed layer. Cosine similarity compares each perturbation direction with the baseline direction.

model	strategy	minimum F1	maximum F1 (layer)	maximum cosine (layer)
Qwen2.5-Coder-1.5B	all_same	1.000	1.000 (L26)	0.334 (L0)
Qwen2.5-Coder-1.5B	baseline	0.905	0.988 (L6)	N/A
Qwen2.5-Coder-1.5B	misleading_iterator	0.991	1.000 (L3)	0.187 (L15)
Qwen2.5-Coder-1.5B	numeric_vars	0.556	0.924 (L23)	0.253 (L19)
Qwen2.5-Coder-1.5B	random_nouns	0.633	0.974 (L9)	0.360 (L16)
Qwen2.5-Coder-1.5B	single_chars	0.664	0.923 (L9)	0.498 (L0)
StarCoder2-7B	all_same	0.999	1.000 (L4)	0.337 (L9)
StarCoder2-7B	baseline	0.898	0.990 (L5)	N/A
StarCoder2-7B	misleading_iterator	0.993	0.999 (L4)	0.265 (L6)
StarCoder2-7B	numeric_vars	0.552	0.958 (L24)	0.248 (L13)
StarCoder2-7B	random_nouns	0.613	0.986 (L7)	0.396 (L8)
StarCoder2-7B	single_chars	0.670	0.961 (L5)	0.451 (L0)

B Causal intervention details

Table 14: Complete gate-specified class/structure patching summary in Qwen2.5-1.5B. The causal gate required mean effect at least 0.10, a positive interval, separation from controls, and recovery at least 0.05 in both directions.

layer	span	direction	control	n	mean effect	95% CI	recovery	minus random
0	query_name	denoise	target	288	+0.0000	[0.0000, 0.0000]	+0.0000	N/A
0	query_name	noise	target	288	+0.0000	[0.0000, 0.0000]	+0.0000	N/A
18	placebo	denoise	target	288	+0.0015	[-0.0010, 0.0043]	+0.0015	N/A
18	placebo	noise	target	288	+0.0006	[-0.0019, 0.0032]	+0.0006	N/A
18	query_name	denoise	random	288	-0.0107	[-0.0205, -0.0014]	-0.0110	N/A
18	query_name	denoise	same_source	288	+0.0004	[-0.0009, 0.0016]	+0.0004	N/A
18	query_name	denoise	target	288	+0.0087	[-0.0007, 0.0188]	+0.0089	+0.0194
18	query_name	noise	random	288	+0.0096	[0.0023, 0.0164]	+0.0099	N/A
18	query_name	noise	same_source	288	-0.0005	[-0.0018, 0.0006]	-0.0006	N/A
18	query_name	noise	target	288	+0.0199	[0.0109, 0.0296]	+0.0204	+0.0103

Table 15: Aggregate probe-link diagnostic for the class/structure evaluation pairs. Absolute values are reported because pair orientation is arbitrary. Large probe movement alongside small behavioral movement is descriptive of the failed causal transfer, not evidence that probe movement causes behavior.

quantity	mean absolute	median absolute
probe movement	15.786	16.302
behavioral movement	0.038	0.031

Table 16: Class/structure gate and readout audit. Both prompt classes favor `True`. The pairwise gap remains separable, but the binary readout is not calibrated to distinguish function prompts.

diagnostic	estimate	95% CI	gate
class logit difference	2.752	[2.624, 2.898]	pass
function logit difference	1.778	[1.652, 1.909]	fail
class-minus-function gap	0.974	[0.908, 1.035]	pass
pair-gap accuracy	0.986	[0.962, 0.993]	pass
query/declaration probe AUC	0.943/0.905	N/A	pass

Table 17: Patching audit trail. Completeness establishes that the scheduled Qwen evaluation finished. It does not turn the failed causal gate into positive evidence. The controller stopped before Coder and StarCoder2 after the Qwen null.

audit item	value
behavior rows present/expected	1152/1152
primary rows present/expected	4032/4032
baseline probe-gap/behavior Spearman	0.164 [0.023, 0.286]
controller terminal state	`stopped_qwen_null`

Status. The iterator recovery exports predate the gate-specified class/structure protocol. They report descriptive recovery curves without clustered confidence intervals or matched placebo/random controls. The terminal value is one by construction, and C-language cross-language rows with zero pairs are missing data rather than null effects.

Table 18: Exploratory iterator activation-patching recovery for Qwen2.5-Coder-1.5B, within-language perturbations.

condition	readout	pairs	clean/corrupt	L1	L2	L3	L4	L5	L6
random_nouns	L6	150	0.999/0.138	0.991	0.930	0.994	0.996	0.998	1.000
single_chars	L6	150	0.996/0.614	0.860	0.799	0.935	0.963	0.983	1.000
all_same	L6	150	0.999/0.142	0.808	0.900	0.961	0.986	0.998	1.000
numeric_vars	L6	150	0.999/0.057	0.966	0.961	0.991	0.997	0.999	1.000
misleading_iterator	L6	150	0.999/0.286	0.820	0.835	0.916	0.946	0.983	1.000

Table 19: Exploratory iterator activation-patching recovery for Qwen2.5-Coder-1.5B, cross-language transfer.

condition	readout	pairs	clean/corrupt	L1	L2	L3	L4	L5	L6	L7
Java	L7	10	0.989/0.709	-0.300	-0.146	0.616	0.867	0.845	0.940	1.000
C++	L7	11	0.979/0.769	-0.066	0.025	0.785	0.869	0.869	0.904	1.000
C	L7	0	0.000/0.000	N/A	N/A	N/A	N/A	N/A	N/A	N/A
C#	L7	9	0.981/0.629	0.307	0.353	0.772	0.867	0.903	0.989	1.000
JavaScript	L7	13	0.983/0.766	0.708	0.678	0.968	0.943	0.953	0.983	1.000
PHP	L7	7	0.982/0.748	-0.211	-0.397	0.614	0.760	0.934	1.027	1.000

Table 20: Exploratory iterator activation-patching recovery for StarCoder2-7B, within-language perturbations.

condition	readout	pairs	clean/corrupt	L1	L2	L3	L4	L5
random_nouns	L5	150	0.998/0.569	0.931	0.766	0.966	0.951	1.000
single_chars	L5	150	0.994/0.686	0.636	0.006	0.823	0.759	1.000
all_same	L5	150	0.999/0.249	0.342	0.772	0.946	0.979	1.000
numeric_vars	L5	150	0.998/0.530	0.889	0.811	0.953	0.955	1.000
misleading_iterator	L5	150	0.997/0.628	0.777	0.104	0.921	0.843	1.000

Table 21: Exploratory iterator activation-patching recovery for StarCoder2-7B, cross-language transfer.

condition	readout	pairs	clean/corrupt	L1	L2	L3	L4	L5	L6	L7	L8	L9	L10
Java	L10	150	0.996/0.750	0.254	0.634	0.829	0.913	0.957	0.964	0.979	0.988	0.994	1.000
C++	L10	150	0.996/0.771	0.337	0.756	0.892	0.938	0.966	0.969	0.979	0.988	0.995	1.000
C	L10	0	0.000/0.000	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
C#	L10	150	0.996/0.740	0.294	0.692	0.877	0.942	0.969	0.974	0.984	0.991	0.996	1.000
JavaScript	L10	150	0.996/0.716	0.311	0.743	0.880	0.939	0.972	0.976	0.983	0.990	0.996	1.000
PHP	L10	73	0.993/0.808	0.491	0.796	0.938	0.964	0.979	0.973	0.983	0.994	0.999	1.000

C Exploratory boolean interventions

Status and estimand. These boolean interventions are exploratory diagnostics rather than evidence for the main causal claim. The committed export contains layer-wise means but not the per-case arrays needed to recompute clustered intervals, and the runs do not use the specified gate of the class/structure experiment. The exporter defines effect as $|\mathrm{clean} - \mathrm{intervened}|$ and specificity as that effect divided by the corresponding random-position-control effect. Specificity can become large when its denominator is small. We therefore report both specificity at the largest-effect layer and the maximum specificity over layers rather than selecting only the more favorable one.

Table 22: Exploratory boolean patch diagnostics. Effect and specificity use the exporter definitions above. No uncertainty interval is available from the committed summary.

language	model	n	peak layer	peak effect	spec. at peak	best spec. (layer)
C++	Qwen2.5-1.5B	157	L0	14.845	3.6×	7.9× (L20)
C++	Qwen2.5-Coder-1.5B	158	L0	15.414	3.6×	6.9× (L8)
C++	StarCoder2-7B	157	L4	14.982	8.1×	10.4× (L16)
Java	Qwen2.5-1.5B	155	L8	11.607	6.1×	7.7× (L12)
Java	Qwen2.5-Coder-1.5B	155	L8	12.642	5.9×	7.7× (L12)
Java	StarCoder2-7B	155	L4	13.900	7.5×	8.6× (L12)
JavaScript	Qwen2.5-1.5B	58	L8	12.755	6.9×	6.9× (L8)
JavaScript	Qwen2.5-Coder-1.5B	62	L8	12.717	6.6×	7.4× (L12)
JavaScript	StarCoder2-7B	129	L4	12.137	8.1×	8.2× (L16)
PHP	Qwen2.5-1.5B	15	L4	11.698	7.2×	7.2× (L4)
PHP	Qwen2.5-Coder-1.5B	16	L4	11.425	4.7×	7.4× (L0)
PHP	StarCoder2-7B	17	L4	11.545	6.3×	9.3× (L16)
Python	Qwen2.5-1.5B	176	L8	12.948	6.0×	7.2× (L16)
Python	Qwen2.5-Coder-1.5B	176	L8	13.275	6.7×	7.8× (L16)
Python	StarCoder2-7B	176	L4	13.592	7.9×	9.7× (L12)

Table 23: Exploratory boolean steer diagnostics. Effect and specificity use the exporter definitions above. No uncertainty interval is available from the committed summary.

language	model	n	peak layer	peak effect	spec. at peak	best spec. (layer)
C++	Qwen2.5-1.5B	79	L24	2.508	45.0×	169.3× (L4)
C++	Qwen2.5-Coder-1.5B	79	L24	1.436	602.0×	602.0× (L24)
C++	StarCoder2-7B	78	L20	0.653	6.9×	345.4× (L4)
Java	Qwen2.5-1.5B	78	L24	2.486	72.2×	826.9× (L8)
Java	Qwen2.5-Coder-1.5B	78	L24	1.712	72.0×	179.9× (L0)
Java	StarCoder2-7B	78	L24	0.555	244.0×	244.0× (L24)
JavaScript	Qwen2.5-1.5B	31	L24	1.763	24.5×	76.8× (L8)
JavaScript	Qwen2.5-Coder-1.5B	31	L24	1.147	65.3×	147.6× (L4)
JavaScript	StarCoder2-7B	65	L16	0.591	11.2×	61.0× (L28)
PHP	Qwen2.5-1.5B	7	L24	2.759	190.2×	190.2× (L24)
PHP	Qwen2.5-Coder-1.5B	8	L24	0.898	11.5×	132.7× (L8)
PHP	StarCoder2-7B	9	L20	0.991	8.7×	99.9× (L4)
Python	Qwen2.5-1.5B	88	L24	2.175	209.1×	794.6× (L16)
Python	Qwen2.5-Coder-1.5B	88	L24	1.231	1027.1×	1027.1× (L24)
Python	StarCoder2-7B	88	L20	0.721	8.6×	55.1× (L4)

Table 24: Exploratory boolean ablate diagnostics. Effect and specificity use the exporter definitions above. No uncertainty interval is available from the committed summary.

language	model	n	peak layer	peak effect	spec. at peak	best spec. (layer)
C++	Qwen2.5-1.5B	157	L0	14.286	4.5×	31.9× (L4)
C++	Qwen2.5-Coder-1.5B	158	L0	12.927	4.4×	76.2× (L20)
C++	StarCoder2-7B	157	L0	13.405	4.3×	36.5× (L4)
Java	Qwen2.5-1.5B	155	L16	9.720	28.5×	86.0× (L12)
Java	Qwen2.5-Coder-1.5B	155	L4	9.331	12.3×	62.4× (L12)
Java	StarCoder2-7B	155	L4	11.190	27.0×	27.0× (L4)
JavaScript	Qwen2.5-1.5B	58	L16	10.852	26.7×	26.7× (L16)
JavaScript	Qwen2.5-Coder-1.5B	62	L16	9.868	28.1×	116.4× (L24)
JavaScript	StarCoder2-7B	129	L4	10.970	27.3×	27.3× (L4)
PHP	Qwen2.5-1.5B	15	L16	15.698	15.0×	102.4× (L20)
PHP	Qwen2.5-Coder-1.5B	16	L16	13.402	26.5×	79.2× (L20)
PHP	StarCoder2-7B	17	L4	14.260	19.4×	19.7× (L8)
Python	Qwen2.5-1.5B	176	L0	12.144	5.8×	36.7× (L24)
Python	Qwen2.5-Coder-1.5B	176	L0	11.286	6.8×	33.1× (L16)
Python	StarCoder2-7B	176	L4	11.818	26.5×	26.5× (L4)

Omitted figures. Per-cell diagnostic plots for the exploratory boolean interventions and the committed role analyses are reproducible from the same artifacts with `make_interp_appendix.py -full-figures`. They visualize the tables above and add no uncertainty estimate the exports lack.

D Explicit coverage holes

- Accumulator and index/key results use legacy exports. No committed problem-grouped, five-seed CSV reproduces those roles under the modern protocol.
- Iterator results are available for Qwen2.5-Coder-1.5B and StarCoder2-7B, but not Qwen2.5-1.5B.
- Class/structure transfer exports include Python, C++, JavaScript, and C, but not Java, C#, or PHP.
- Boolean probe exports include Python, Java, JavaScript, and PHP. C, C#, and C++ are absent, and renaming is available only for Python.
- Gate-specified class/structure patching stopped after the Qwen2.5-1.5B null. Coder, StarCoder2, and the all-layer core sweep were not attempted.
- No matched model-free surface comparator is committed for the class/structure role.

E Reproducibility

The appendix is generated from committed boolean, iterator, class or structure, and patching artifacts by `scripts/make_interp_appendix.py`. Modern probe conditions use frozen problem hashes, validation-selected layers, and five seeds where stated. Iterator summaries are single-run. The class or structure intervention records model and dataset revisions, prompt and configuration hashes, all 4,032 primary rows, and clustered intervals. A regression test checks source traces for the headline numbers and rejects LP4FM-only conditions in this manuscript.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction distinguish decodability, lexical robustness, surface irreducibility, and causal use. They state that the study includes three models, five roles, and seven languages collectively without claiming a complete factorial design.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The Limitations section covers uneven model and language coverage, legacy split and renaming weaknesses, label and surface confounds, and the one-model, one-site scope of the causal null.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper reports no theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The study-design section reports the full evidence matrix and separates modern problem-grouped experiments from legacy occurrence-level runs. The reproducibility appendix records artifact, model-revision, split, seed, and intervention provenance.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: Result files and analysis scripts are prepared for release, but the manuscript does not yet provide an anonymous review URL or a complete archival package.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The study-design section specifies pooling, linear probes, control types, split units, layer selection, and which role experiments use the modern five-seed protocol. Legacy protocol differences are stated explicitly.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: The class-reference probe and causal intervention report clustered intervals, and the boolean analysis reports the smallest resolvable macro-F1 step. Accumulator, index, and iterator headline deltas do not have commensurate intervals, so uncertainty remains incomplete.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [No]

   Justification: The paper does not report hardware, memory, or complete wall-clock costs.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work analyses publicly released models on a public benchmark of competitive-programming solutions. It involves no human subjects, no personally identifying data, and no deployment.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

Justification: The work is an analysis of how existing code models represent variable roles. We identify no societal impact specific to it beyond that of the public models and benchmark it studies.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper releases no data or models. It analyses publicly available ones.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No]

Justification: XLCoST, both Qwen releases, and StarCoder2 are cited by name, but the manuscript does not enumerate every asset's license name and terms.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.

- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [No]

    Justification: The analysis scripts and result files have artifact provenance, but a complete user-facing release package and documentation are not yet included with the anonymous submission.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper involves no crowdsourcing and no human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper involves no human subjects, so no IRB review was required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: Large language models are the object of study, not an important or non-standard component used to develop the research method. Writing and editing assistance is exempt under the question.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.